

Full Stack Take-Home

Preparation Guide

This guide exists because we want you to do well. We have reviewed enough submissions to know exactly where candidates succeed and where they fall short. The information below reflects that directly. Read it carefully before you write a single line of code.

This is the only hands-on technical interview in our process so it carries a lot of weight.

What This Exercise Is Actually Testing

The assignment is a small task management application. What we are evaluating is not how many features you can build. It is whether your judgment matches the size of the problem.

The word senior gets used a lot in job descriptions. In this exercise, it means one specific thing: can you read a problem, match your solution to it, finish what you start, and tell the truth about what you built?

Every decision you make, from how you structure the project to what you write in the README, tells us something about how you work. **Treat this the way you would treat a pull request that a teammate who has never met you will review on their first day.**

Ask yourself before submitting: if a colleague cloned this repo on their first day, could they understand what it does, set it up, and run it without messaging you for help? If the answer is no, your submission is not finished.

The Two Ways Submissions Fail

Most rejections come from one of two failure modes. They look different on the surface but share the same root cause: the submission does not deliver a finished product.

Failure Mode 1: You Built a Demo

Under-built submissions look like a proof of concept, not a working application. The core mistake is that the candidate made scope decisions and then did not finish what they chose to build.

Signals that a submission is a demo:

- Data disappears when you restart the app. A filing cabinet that empties itself is not a filing cabinet.

- The app accepts bad input Empty titles. Nonsense dates. No feedback when something fails.
- The backend supports delete but there is no delete button. The backend supports edit but there is no edit form. Features that exist in one place and nowhere else are not features.
- Login exists but any logged-in user can access any other user's data. A lock on the lobby with no locks on the apartments is not security. It is worse than no lock at all because it creates a false sense of protection.
- Error cases are invisible. The happy path works. Everything else crashes or disappears silently.

The rule: every feature you include is a claim that it works, end to end. A half-built feature counts against you more than a feature you honestly chose not to build.

Failure Mode 2: You Built a Beautiful Structure Around a Broken Product

Over-built submissions have the opposite problem. The architecture is elaborate. The code is clean. And the product inside it is incomplete.

This is fundamentally a time allocation problem. You had six hours. You spent four of them building scaffolding that does not make the product better. You had two hours left to build a working app. That is not enough.

Signals that a submission is over-engineered for this scope:

- Multiple backend projects for a single set of CRUD endpoints. One project is the right shape.
- A repository layer that wraps the database with exactly one implementation. An abstraction that never gets a second use is not an abstraction. It is complexity.
- Patterns like CQRS or MediatR applied to an app with one entity and a handful of endpoints. These tools exist for specific problems. This is not one of them.
- Architectural decision records, CI pipelines and observability infrastructure for a six-hour exercise. This signals that you ran out of time to finish the product.

The rule: every layer you add is a claim that this app needs that layer. On a task manager, most of those claims are false. Reviewers will read unnecessary structure as poor judgment, not as ambition.

The Full Circle Rule

Before you submit, trace every feature you built from the user's action all the way around. Start at what the user does, follow it through the frontend, through the backend, into the database, back out, and then look at what the user sees.

If any point in that circle is broken or missing, you have not shipped that feature.

Login exists. Where does it go after the user signs in?

Delete exists in the backend. Where is the button in the UI? When the user clicks it, does the list update immediately? What does the user see if the delete fails?

Create exists. Is there a form? Does it validate before submission? What happens when validation fails? Does the form keep the user's input or erase it?

Edit exists. Is there an edit form? Does it populate with the current values? What happens on a validation error?

The everyday flows matter too. Due dates should behave correctly regardless of timezone. If you build them, build them right. Lists should update immediately after a create, edit, or delete without requiring a page refresh. These feel small individually. Together they tell the reviewer whether you actually used what you built.

Authentication and Ownership

Authentication is expected. If you build it, you must close the loop.

If you choose not to implement authentication:

This is a legitimate call. Some submissions pass without it. But you must be explicit in your README: Silence reads as an oversight. A clear explanation reads as judgment.

Tests

Tests are expected. Not comprehensive coverage. Not a test for every line. The right question to ask yourself is: what would I be most nervous about if someone else changed this code?

For this application, the highest-risk areas are almost always the same two things:

- **Ownership enforcement.** Can User A access User B's data? This should be tested explicitly.
- **Validation.** Does the application correctly reject bad input? Empty titles. Missing due dates. Invalid values.

A small number of real tests on these two areas signals better judgment than twenty tests that verify a button renders. No tests, or a test project with no actual assertions, is one of the most common written rejection reasons in our review history.

The README

Your README is doing more work than you think. Reviewers read it carefully and then check whether the code matches it. A mismatch is not a writing issue. It is a trust signal.

What a passing README looks like:

- How to start the application. Clear and accurate.
- What you built. Honest and specific.
- What you deliberately left out and why. This is a senior trait. Pretending something is complete when it is not is not.
- What you would do with another day. This shows you understand the gap between what you built and what a production version would require.

What a failing README looks like:

- Claims the submission is production-ready when core features are missing or broken.
- Documents features that do not exist in the code.
- Is the default template from the frontend scaffold with nothing changed.
- Says nothing about authentication when the app has none.

Short and honest beats long and impressive. Overclaiming costs you more than missing features would.

What We Are Not Asking For

Do not spend your time on these. They are not evaluated and building them takes time away from finishing your features.

- CI/CD pipelines
- Docker or containerization (fine to include if it helps you, not required)
- Deployment configuration
- Observability or monitoring infrastructure
- Visual polish beyond clear and usable

Before You Submit

Step back and review your submission as if you were the reviewer who has never met you.

- Clone your repo fresh and follow your own README. Does it actually work?
- Trace every feature around the full circle. Is every feature complete on both sides?
- Click through every flow. Do error cases have a real user-facing response?
- Check your repository for anything that should not be there.
- Read your README. Does it describe what the application actually does, not what you intended it to do?
- If you implemented authentication, verify that User A cannot access User B's data through any API call.
- **We are not counting features. We are looking at whether the features you chose to build are finished end to end**

Good luck. We look forward to reading your work.